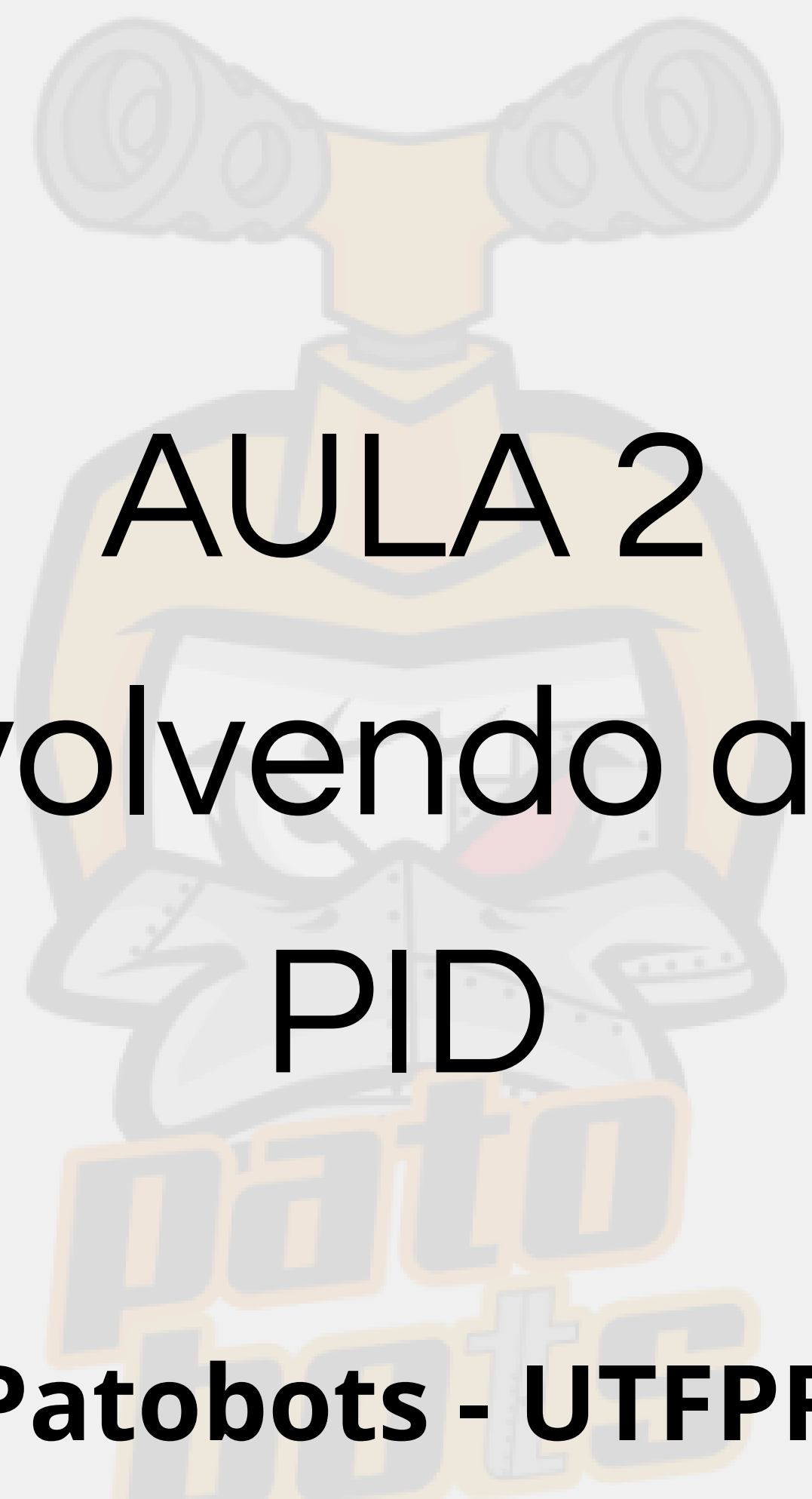




AULA 2

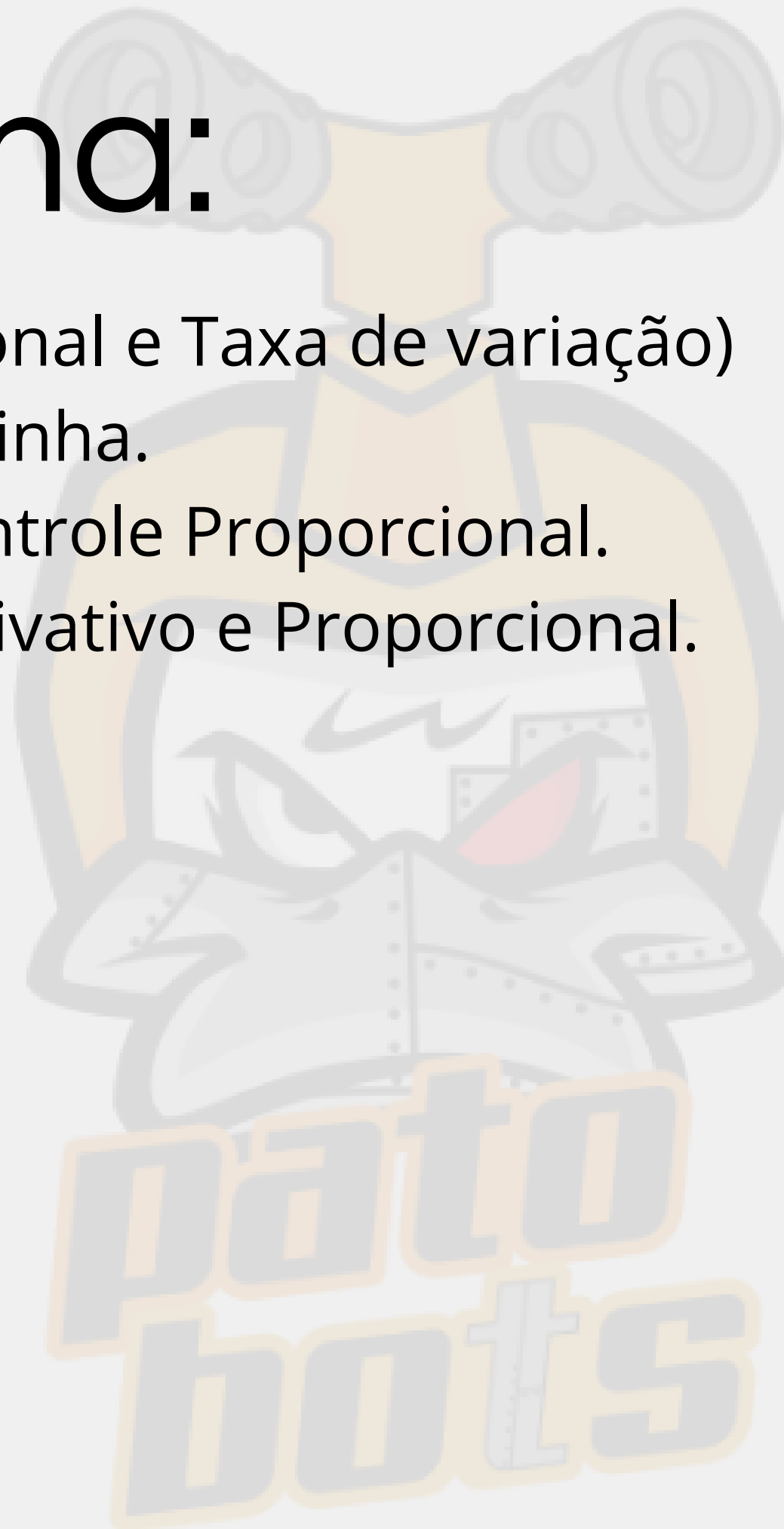
Desenvolvendo a Lógica

PID

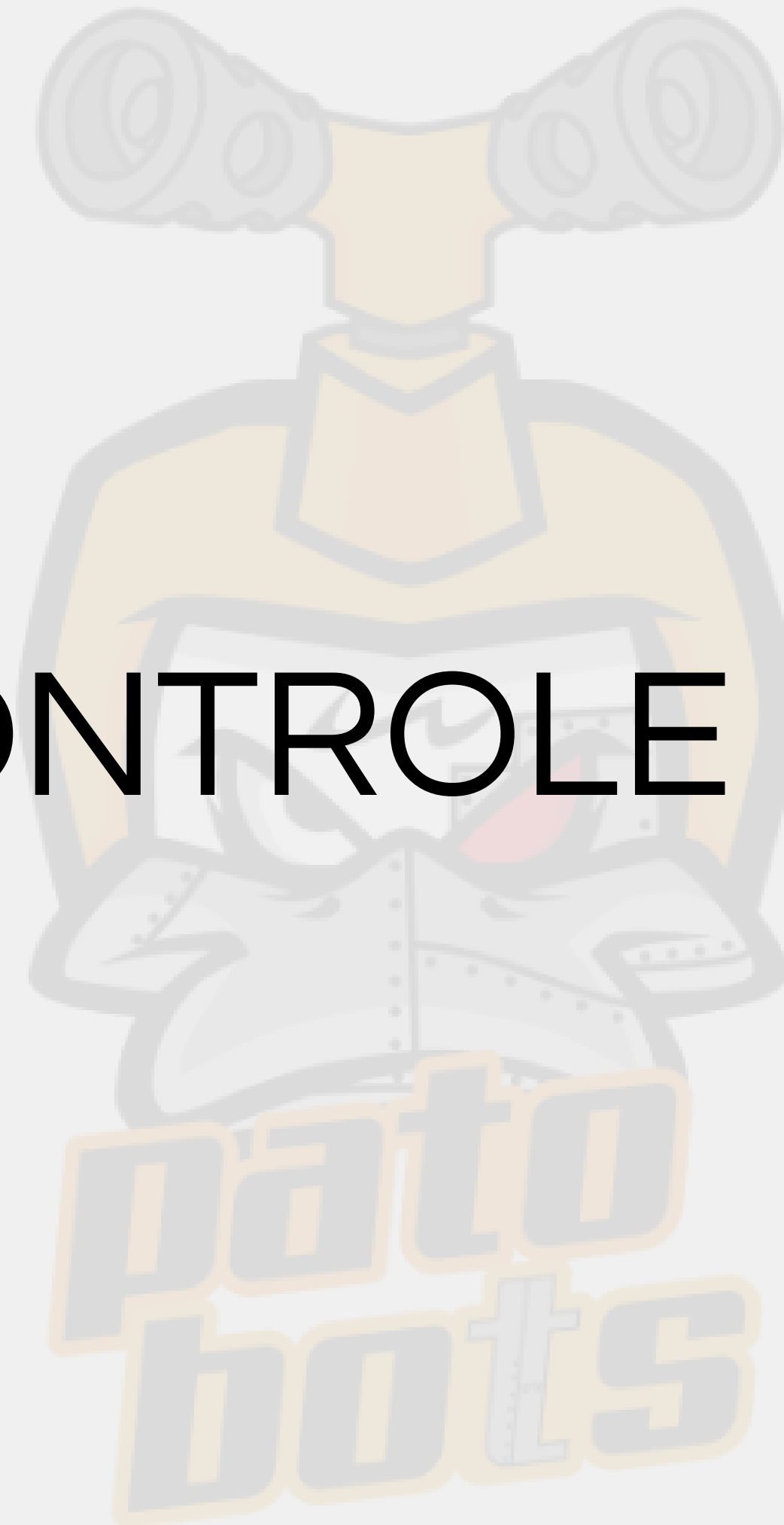
Patobots - UTFPR

Cronograma:

- Controle PID. (Proporcional e Taxa de variação)
- Lógica do Seguidor de Linha.
- Teste com apenas o Controle Proporcional.
- Teste com Controle Derivativo e Proporcional.

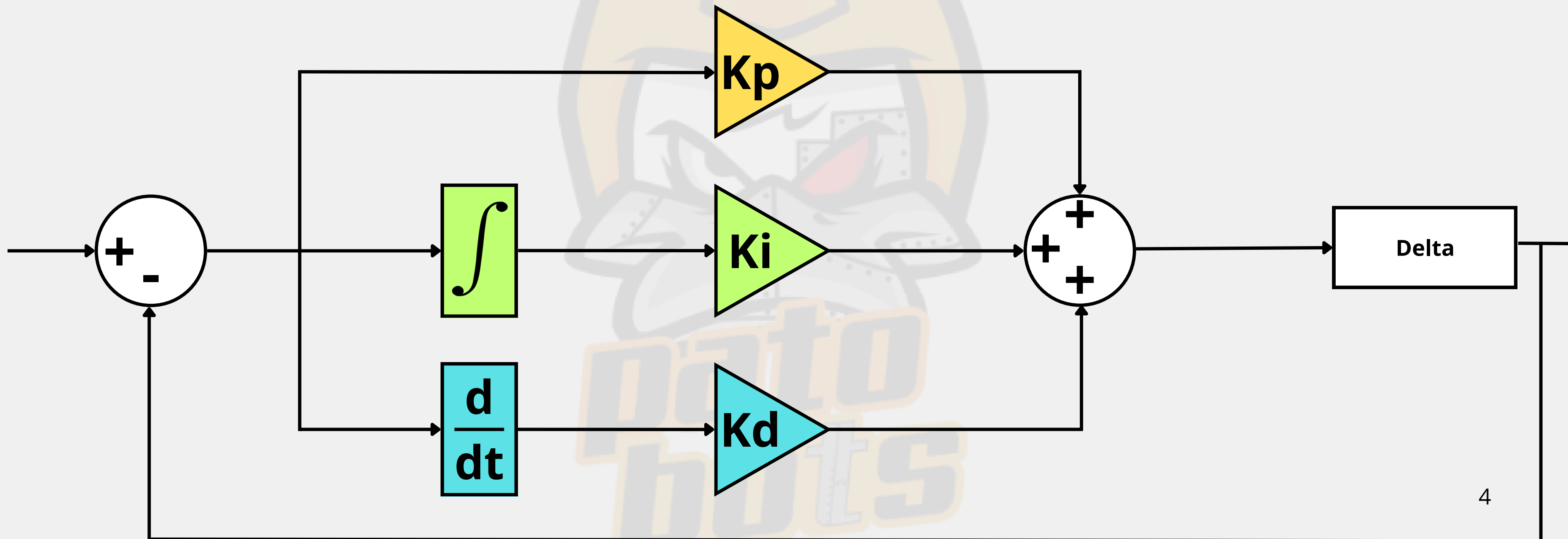


CONTROLE PID



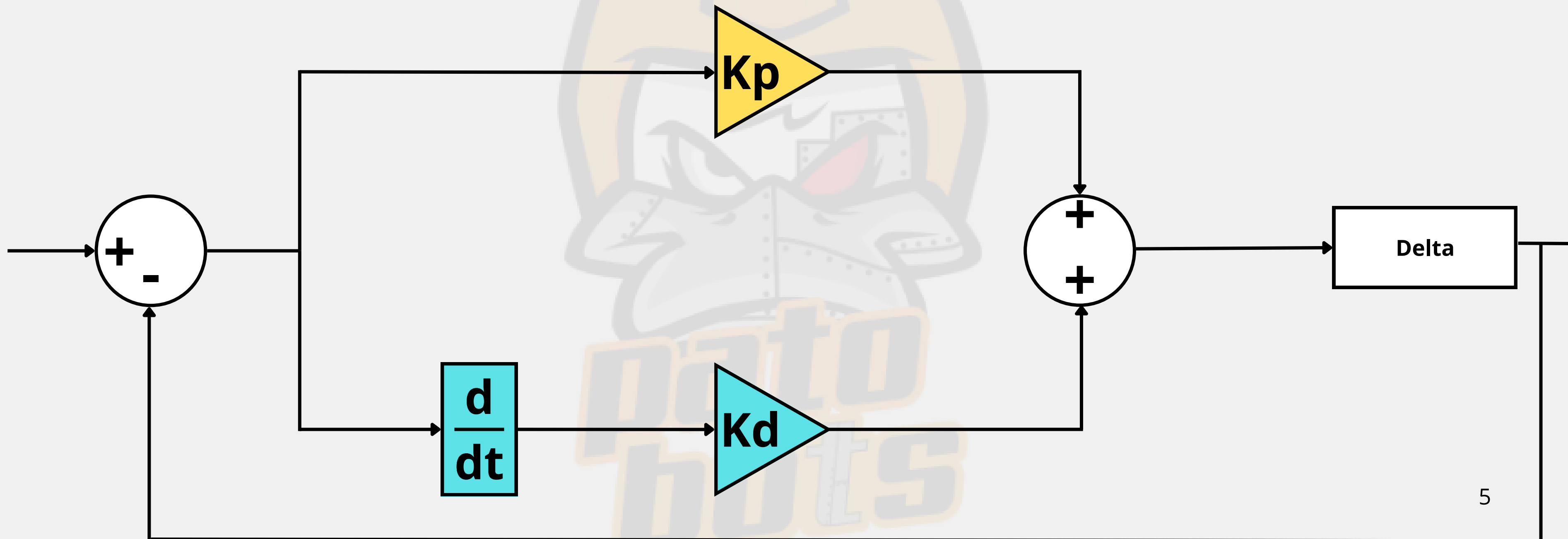
Controle PID

(Proporcional, Integral e Derivativo)



Controle PID

(Proporcional, Integral e Derivativo)



Controle PID

Controle PID para Robôs:

- **Estabilização:** O Controle ajuda o robô a manter equilíbrio e seguir o caminho correto.
- **Seguidor de Linha:** Ideal para robôs que precisam seguir trajetórias marcadas no chão.



Controle PID

Fórmula de Controle P e D:

Ajusta a movimentação do robô com base no erro de posição:

- **P (Proporcional):** Reage ao erro atual.
- **D (Derivativo):** Considera a variação do erro para corrigir a velocidade.

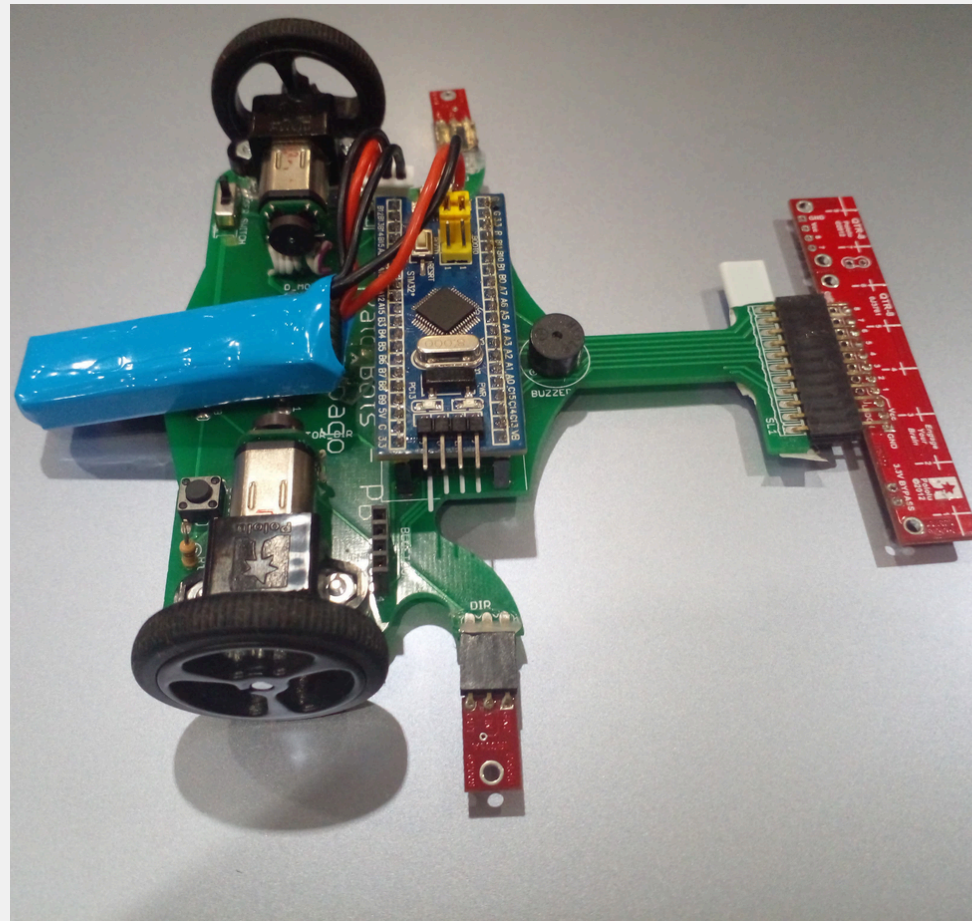
- **Fórmula:**

$$\text{Delta} = K_p * \text{Erro Atual} + K_d * (\text{Erro Atual} - \text{Erro Anterior})$$

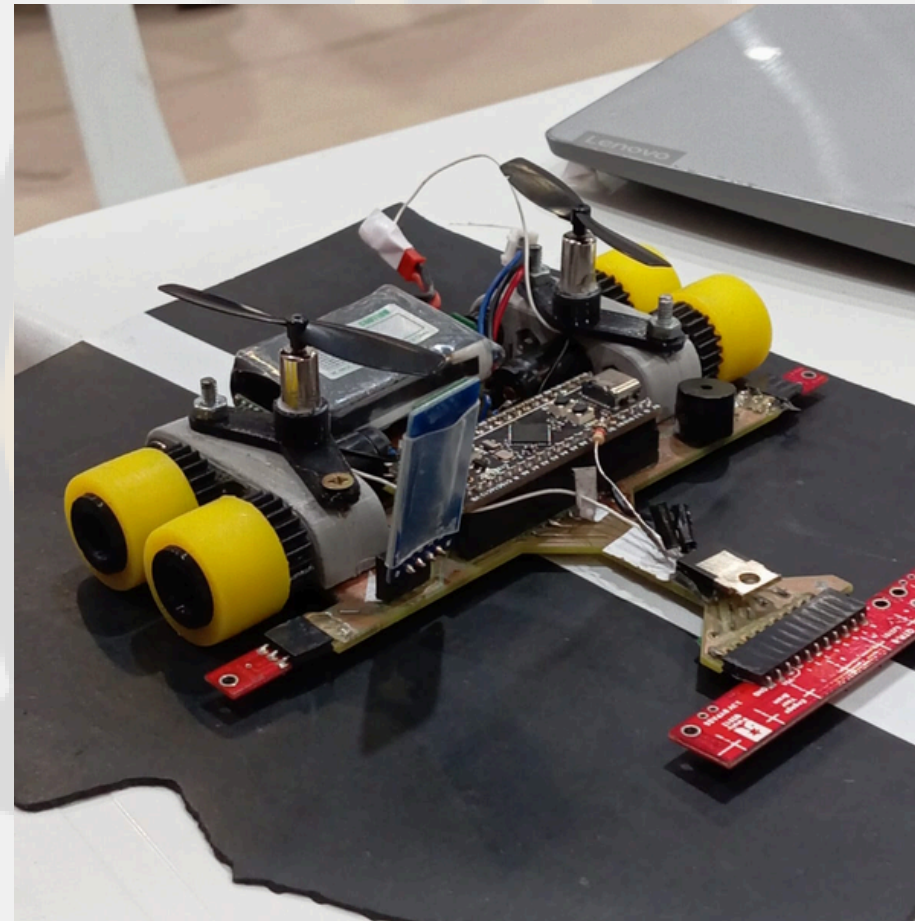
SEGUIDOR DE LINHA



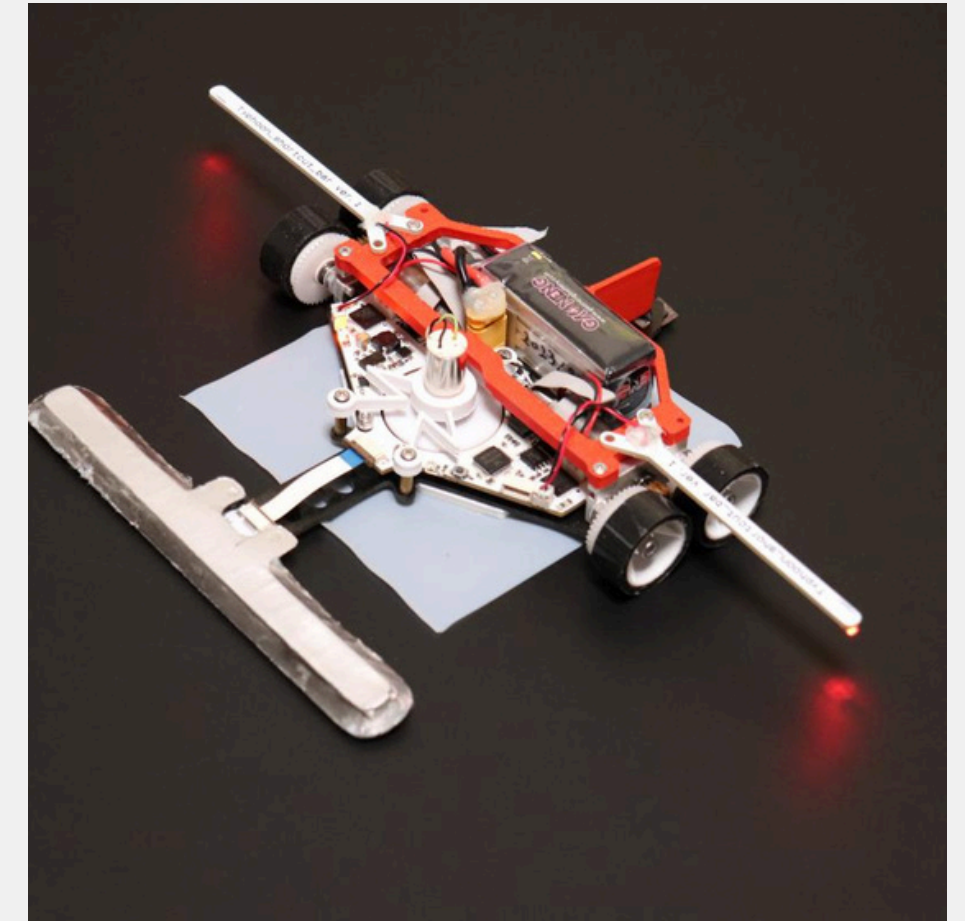
Tipos de Seguidor de Linha PRO



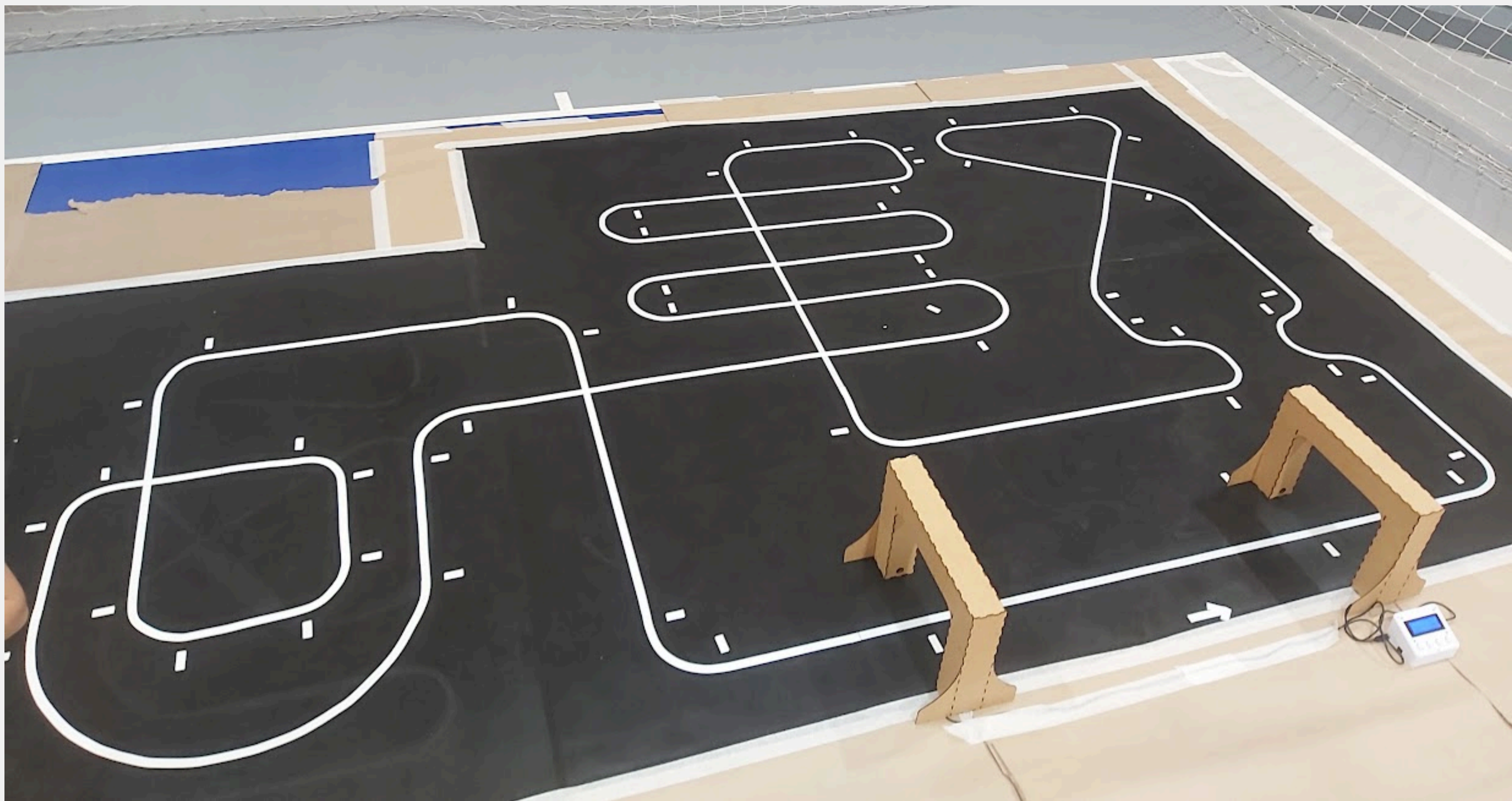
Básico



Com Ventoinha



Com Sucção



Lógica do Seguidor de Linha

- O robô detecta uma faixa no chão e ajusta sua trajetória para permanecer sobre ela.
- Seus sensores identificam a posição da linha e enviam sinais para o **Sistema de Controle**.
- O **Sistema de Controle** ajusta os motores com base nos dados dos sensores, corrigindo sua direção.





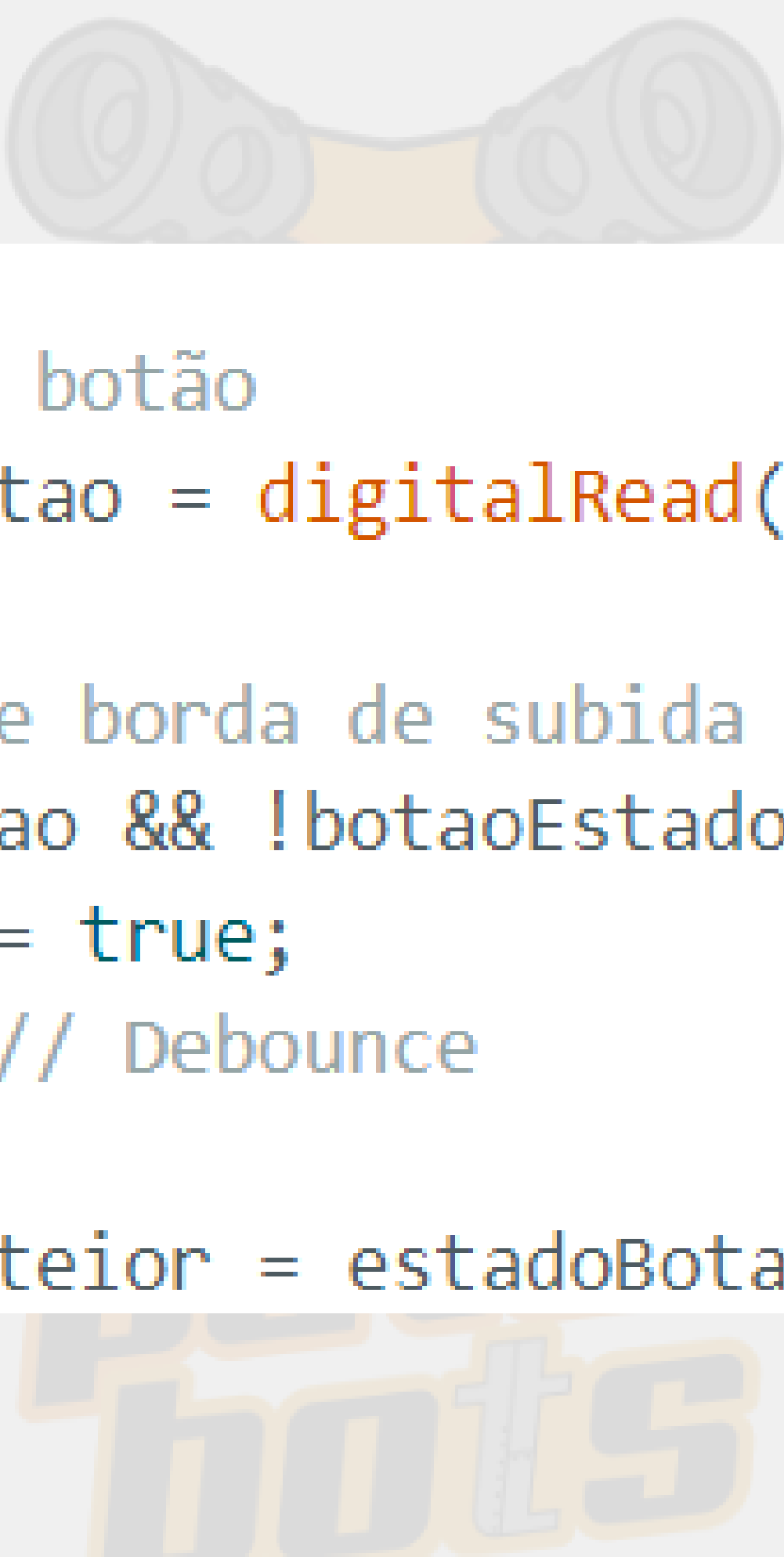
CÓDIGO DE CONTROLE DO SEGUIDOR (Proporcional)

```
6 // Definições dos pinos
7 #define NUM_SEN 8
8 #define BUTTON_PIN 12
9
10 // Pinos do motor driver TB6612FNG
11 #define PWMA 5 // PWM Motor A (esquerda)
12 #define AI1 9 // Direção Motor A
13 #define AI2 4 // Direção Motor A
14 #define PWMB 6 // PWM Motor B (direita)
15 #define BI1 7 // Direção Motor B
16 #define BI2 8 // Direção Motor B
17
18 // Pinos dos sensores (A0 a A7)
19 int pinoSensores[NUM_SEN] = {A0, A1, A2, A3, A4, A5, A6, A7};
20
21 // Variáveis para calibração
22 int minSensor[NUM_SEN];
23 int maxSensor[NUM_SEN];
24 int valorSensores[NUM_SEN];
25
26 // Variáveis para controle PID
27 float Kp = 0.0; // Constante proporcional
28 float erro = 0;
```

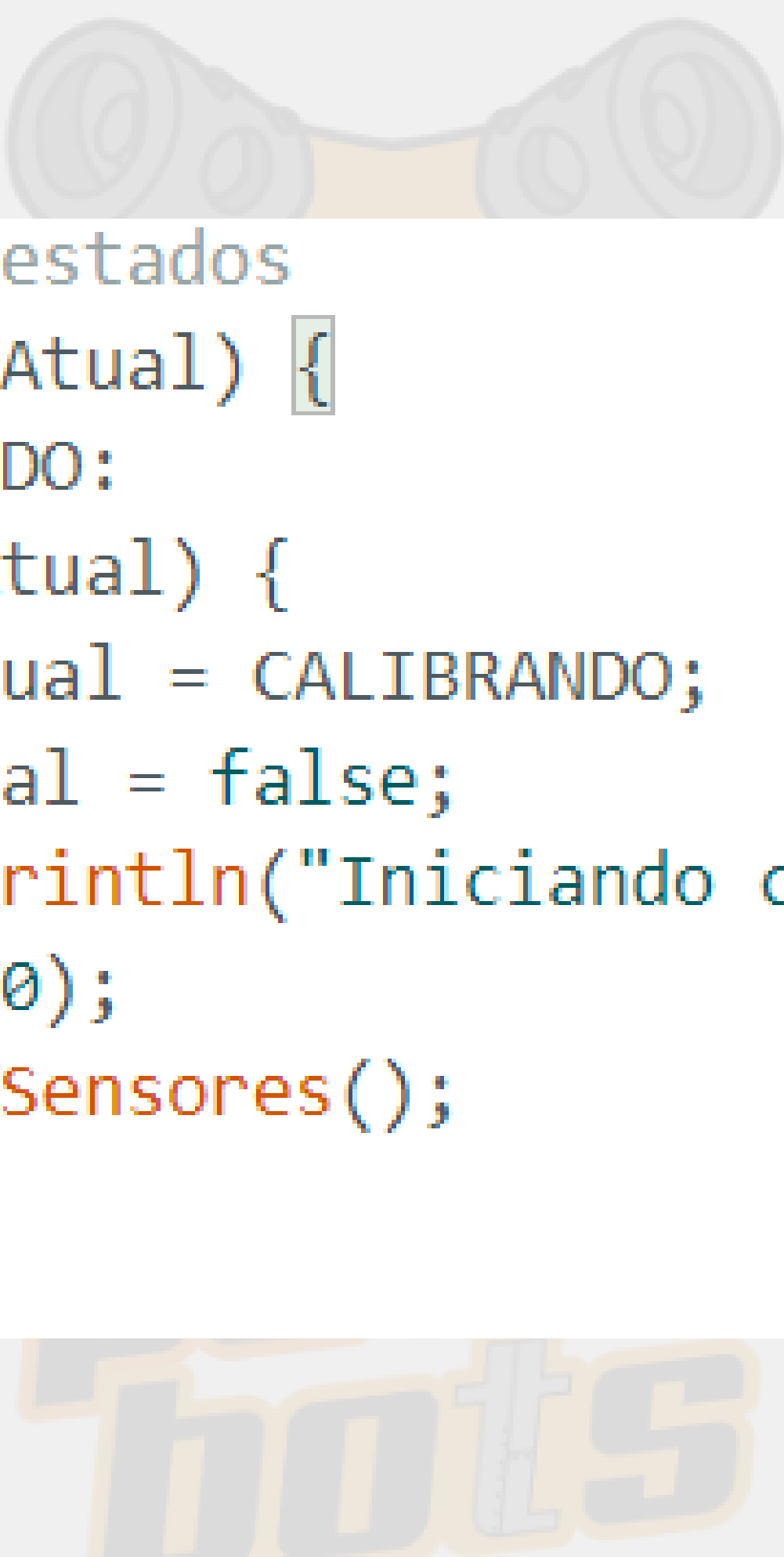
```
30 // Velocidades dos motores
31 int velBase = 40; // Velocidade base (0-255)
32 int velMax = 255; // Velocidade máxima
33 int velMin = 0; // Velocidade mínima
34
35 // Estados do programa
36 enum Estado {
37     ESPERANDO,
38     CALIBRANDO,
39     CORRENDO
40 };
41
42 Estado estadoAtual = ESPERANDO;
43 bool botaoAtual = false;
44 bool botaoEstadoAnterior = false;
45
```



```
48 void setup() {
49     Serial.begin(9600);
50
51     // Configuração dos pinos
52     pinMode(BUTTON_PIN, INPUT);
53
54     // Configuração dos pinos do motor
55     pinMode(PWMA, OUTPUT);
56     pinMode(AI1, OUTPUT);
57     pinMode(AI2, OUTPUT);
58     pinMode(PWMB, OUTPUT);
59     pinMode(BI1, OUTPUT);
60     pinMode(BI2, OUTPUT);
61
62     // Parar motores inicialmente
63     pararMotores();
64
65     // Inicializar valores de calibração
66     for (int i = 0; i < NUM_SEN; i++) {
67         minSensor[i] = 1023;
68         maxSensor[i] = 0;
69     }
70
71     Serial.println("Sistema iniciado. Pressione o botão para calibrar.");
72 }
```



```
75 void loop() {  
76     // Leitura do botão  
77     bool estadoBotao = digitalRead(BUTTON_PIN);  
78  
79     // Detecção de borda de subida do botão  
80     if (estadoBotao && !botaoEstadoAnterior) {  
81         botaoAtual = true;  
82         delay(50); // Debounce  
83     }  
84     botaoEstadoAnterior = estadoBotao;
```

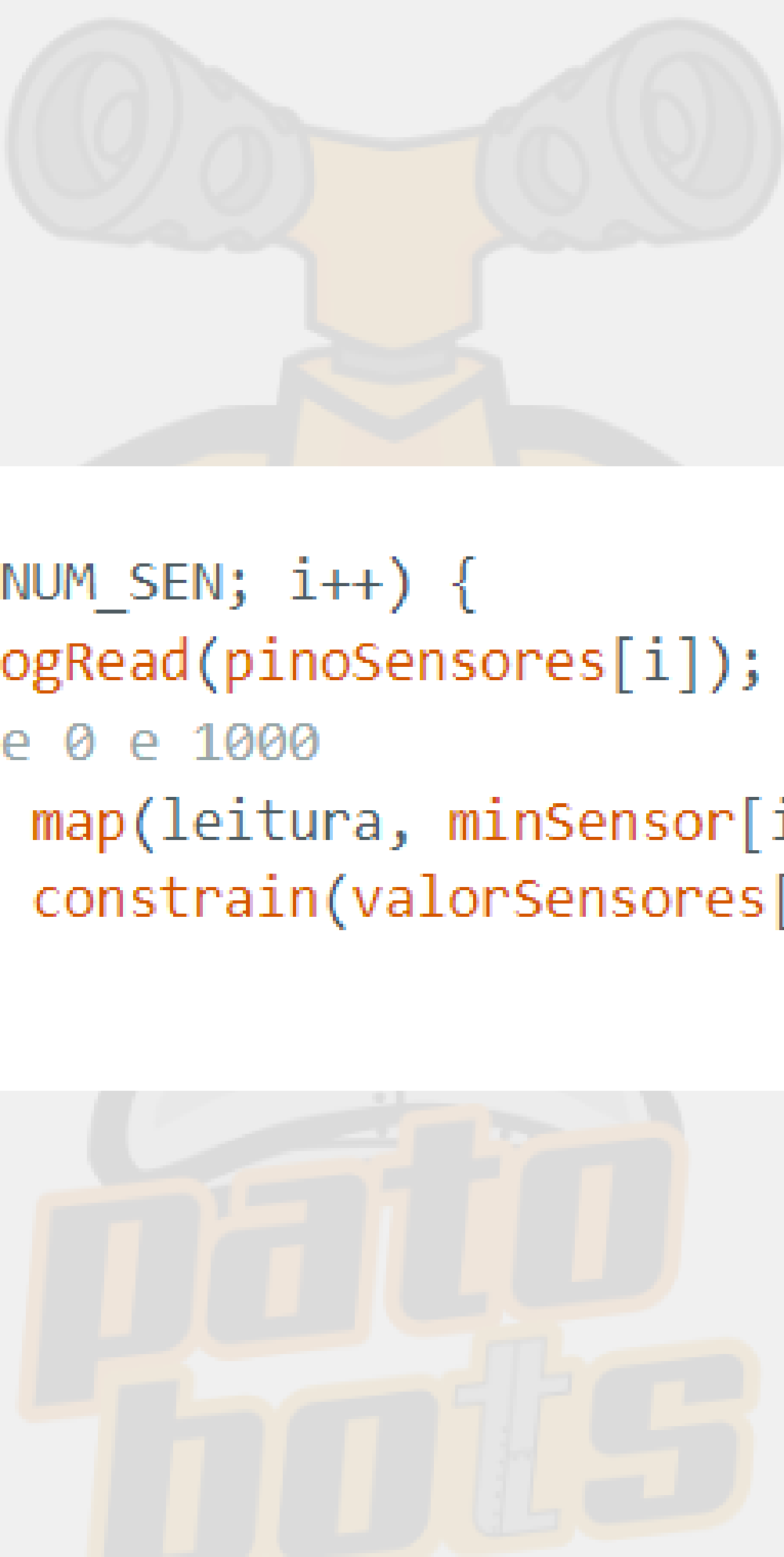
```
86 // Máquina de estados
87 switch (estadoAtual) {
88     case ESPERANDO:
89         if (botaoAtual) {
90             estadoAtual = CALIBRANDO;
91             botaoAtual = false;
92             Serial.println("Iniciando calibração...");
93             delay(500);
94             calibrarSensores();
95         }
96         break;
```

```
98     case CALIBRANDO:
99         estadoAtual = CORRENDO;
100         Serial.println("Calibração concluída. Iniciando corrida!");
101         delay(3000); // Depois de 3s inicia a corrida
102         break;
103
104     case CORRENDO:
105         if (botaoAtual) {
106             estadoAtual = ESPERANDO;
107             botaoAtual = false;
108             pararMotores();
109             Serial.println("Parando robô. Pressione o botão novamente para r
110         } else {
111             seguirLinha();
112         }
113         break;
114     }
115 }
```

```
117 void calibrarSensores() {
118     Serial.println("Movendo robô para calibração...");
119
120     // Calibração por 3 segundos girando o robô
121     unsigned long tempoInicial = millis();
122
123     while (millis() - tempoInicial < 3000) {
124         // Girar o robô lentamente para a direita
125         acelerar(50, -50);
126
127         // Ler sensores e atualizar valores min/max
128         for (int i = 0; i < NUM_SEN; i++) {
129             int faixa = analogRead(pinoSensores[i]);
130             if (faixa < minSensor[i]) {
131                 minSensor[i] = faixa;
132             }
133             if (faixa > maxSensor[i]) {
134                 maxSensor[i] = faixa;
135             }
136         }
137         delay(10);
138     }
```

```
140 // Girar para o outro lado
141 tempoInicial = millis();
142 while (millis() - tempoInicial < 3000) {
143     // Girar o robô lentamente para a esquerda
144     acelerar(-50, 50);
145
146     // Ler sensores e atualizar valores min/max
147     for (int i = 0; i < NUM_SEN; i++) {
148         int faixa = analogRead(pinoSensores[i]);
149         if (faixa < minSensor[i]) {
150             minSensor[i] = faixa;
151         }
152         if (faixa > maxSensor[i]) {
153             maxSensor[i] = faixa;
154         }
155     }
156     delay(10);
157 }
158
159 pararMotores();
```

```
71 void seguirLinha() {
72     // Ler e normalizar sensores
73     lerSensores();
74
75     // Calcular posição da linha
76     float posicao = controleLinha();
77
78     // Calcular erro (0 = centro, negativo = esquerda, positivo = direita)
79     erro = posicao - 3.5; // Centro dos 8 sensores (0-7)
80
81     // Controle PID
82     float pid = Kp * erro;
83
84     int velEsquerda = pid;
85     int velDireita = -pid;
86
87     // Limitar velocidades
88     velEsquerda = constrain(velEsquerda, velMin, velMax);
89     velDireita = constrain(velDireita, velMin, velMax);
90
91     // Aplicar velocidades aos motores
92     acelerar(velEsquerda, velDireita);
93 }
```


The logo for 'Pato Bots' is centered in the background. It features a stylized robot head with two large, circular, metallic-looking eyes and a yellow body. Below the head, the words 'pato' and 'bots' are written in a bold, rounded, sans-serif font. The 'pato' is in a light blue color, and 'bots' is in a darker blue, with a small robot arm integrated into the letter 't' of 'bots'.

```
95 void lerSensores() {  
96     for (int i = 0; i < NUM_SEN; i++) {  
97         int leitura = analogRead(pinoSensores[i]);  
98         // Normalizar entre 0 e 1000  
99         valorSensores[i] = map(leitura, minSensor[i], maxSensor[i], 0, 1000);  
100        valorSensores[i] = constrain(valorSensores[i], 0, 1000);  
101    }  
102 }
```

```
104 float controleLinha() {
105     long num = 0;
106     long den = 0;
107
108     for (int i = 0; i < NUM_SEN; i++) {
109         num += (long)valorSensores[i] * i * 1000;
110         den += valorSensores[i];
111     }
112
113     if (den == 0) {
114         return 3.5; // Retorna centro se nenhum sensor detectar linha
115     }
116
117     return (float)num / den / 1000.0;
118 }
```



```
120 void acelerar(int velEsquerda, int velDireita) {
121     // Motor esquerdo (canal A)
122     if (velEsquerda >= 0) {
123         digitalWrite(AI1, HIGH);
124         digitalWrite(AI2, LOW);
125         analogWrite(PWMA, velEsquerda);
126     } else {
127         digitalWrite(AI1, LOW);
128         digitalWrite(AI2, HIGH);
129         analogWrite(PWMA, -velEsquerda);
130     }
131
132     // Motor direito (canal B)
133     if (velDireita >= 0) {
134         digitalWrite(BI1, HIGH);
135         digitalWrite(BI2, LOW);
136         analogWrite(PWMB, velDireita);
137     } else {
138         digitalWrite(BI1, LOW);
139         digitalWrite(BI2, HIGH);
140         analogWrite(PWMB, -velDireita);
141     }
142 }
```

```
144 void pararMotores() {  
145     analogWrite(PWMA, 0);  
146     analogWrite(PWMB, 0);  
147     digitalWrite(AI1, LOW);  
148     digitalWrite(AI2, LOW);  
149     digitalWrite(BI1, LOW);  
150     digitalWrite(BI2, LOW);  
151 }
```



CÓDIGO DE CONTROLE DO SEGUIDOR (Derivativo)



```
26 // Variáveis para controle PID
27 float Kp = 0.0;    // Constante proporcional
28 float Kd = 0.0;    // Constante diferencial
29 float erro = 0;
30 float erroAnterior = 0;
31
```



```
73 void seguirLinha() {
74     // Ler e normalizar sensores
75     lerSensores();
76
77     // Calcular posição da linha
78     float posicao = controleLinha();
79
80     // Calcular erro (0 = centro, negativo = esquerda, positivo = direita)
81     erro = posicao - 3.5; // Centro dos 8 sensores (0-7)
82
83     // Controle PID
84     float derivativo = erro - erroAnterior;
85     float pid = Kp * erro + Kd * derivativo;
86     erroAnterior = erro;
87
88     int velEsquerda = pid;
89     int velDireita = -pid;
90
91     // Limitar velocidades
92     velEsquerda = constrain(velEsquerda, velMin, velMax);
93     velDireita = constrain(velDireita, velMin, velMax);
94
95     // Aplicar velocidades aos motores
96     acelerar(velEsquerda, velDireita);
97 }
```